Math.Comput.Sci. (2020) 14:551–558
https://doi.org/10.1007/s11786-020-00451-7

Mathematics in Computer Science

# Image Analysis: Identification of Objects via Polynomial Systems

**Robert H. Lewis**

Received: 1 November 2018 / Revised: 24 June 2019 / Accepted: 15 December 2019 / Published online: 7 February 2020


**Abstract** The problem is to identify a movable object that is in some sense known, if it is encountered later. Suppose we have a sensor, on a fixed radar station or a moving platform. We have an object, say object A, previously measured, with certain distinct identifiable points $p_i$. We know the distances between these points. We later encounter a similar object B and want to know if it is A. We have a sensor that sends and receives electronic signals, and so we measure the distances $t_i$ from the sensor to the distinguished points on B. We first consider the two-dimensional case. Assume there are three distinct points on A. We have our measured distances $t_1, t_2, t_3$ and previously known distances between the points on A, $d_1, d_2, d_3$. We derive a polynomial system relating these quantities and show that it is easy to solve yielding a *resultant* that is the "signature" for A. Its use will eliminate B if B is not A. The generalization to three dimensions is immediate. We need a fourth point. The polynomial system contains many parameters, but we solve it symbolically. We then discuss generalizations involving flexibility. In those cases we need five points and the systems are much more complex. In section five we consider the same question of recognition but using a different type of sensor, one that can only receive. We compare a Gröbner basis approach with the Dixon resultant method. We compare solutions on *Magma*, *Maple*, and *Fermat* computer algebra systems.

**Keywords** Image analysis · Bistatic radar · Polynomial system · Resultant · Parameters · Dixon · Gröbner basis

**Mathematics Subject Classification** 13P15 · 65D18 · 68U05 · 68U10

## 1 Introduction

Identifying an object if it moves or if the perspective changes is a classic problem in image recognition and computer vision [9,13]. In [9] we considered and solved the so-called "Six-Line Problem", in which a 3D object like a building has six distinguished lines. The object is considered "known" via these lines. The problem is to decide if a similar object encountered later from another viewpoint is in fact the same object. Via techniques of algebraic geometry, a system of polynomial equations was derived. The *variables* in these equations were the transformation coordinates, and the *parameters* specified the characteristics of the lines. There were four equations in three variables and 13

Robert H. Lewis (✉)
Fordham University, New York, USA
e-mail: rlewis@fordham.edu

parameters. The equations were solved *symbolically*, i.e., the thirteen parameters are retained as symbolic names. The *resultant* [2,14], a single polynomial in 239 terms, was computed with the Dixon-KSY method [6,8]. To use this on a test object, numerical values for its 13 parameters would be substituted. If the result is not 0, this is not the original object. If it is 0, it is highly likely to be the original object.

In this work we consider a similar but actually simpler situation. Suppose we have a sensor, on a fixed radar station or a moving platform. We have an object, say object A, previously measured, with certain distinct identifiable points $p_i$. We know the distances between these points, $d_i$. We later encounter a similar object B and want to know if it is A. The sensor sends and receives electronic signals, and so we measure the distances $t_i$ from the sensor to the distinguished points on B.

We first consider the two-dimensional case. Assume there are three distinct points on A. We have our measured distances $t_1, t_2, t_3$ and previously known distances between the points on A, $d_1, d_2, d_3$. We derive a polynomial system relating these quantities and show that it is easy to solve yielding a resultant that is the "signature" for A. Its use will eliminate B if B is not A.

The generalization to three dimensions is immediate. We need a fourth point. The polynomial system contains many parameters, but we solve it symbolically. We then discuss generalizations involving flexibility. In those cases we need five points and the systems are much more complex.

In section five we consider the same question of recognition but using a different type of sensor, one that can only receive. A separate transmitter sends out the beam. This situation may be referred to as bistatic sonar or bistatic radar [5,15].

## 2 Polynomial Systems

The theory of eliminating variables from a system of equations has a long history, starting with Bezout around 1760. A key idea is the *resultant* of a system of polynomial equations [2,14]. Bezout did this for one-variable polynomials. Dixon [3] extended it to multivariate polynomials, and proved it would work in a certain ideal situation. However, for real problems the ideal situation rarely applies and often the method seems to fail. Kapur, Saxena, and Yang showed how to get around all those problems in 1994 [1,6]. Lewis refined and greatly improved the method in 2008 [8] to what is called Dixon-EDF. See [7] for an explanation of the method. In early 2019 Lewis developed further acceleration techniques [12], one of which (block decomposition) has been used in the problems here. Gröbner bases can also be used to eliminate variables [14].

Here is a short description of Dixon-KSY-EDF. Given a set of $n$ multivariate polynomials, certain clever substitutions are done with $n-1$ new symbolic variables. An $n \times n$ matrix is created and its determinant computed. This is called the *Dixon polynomial*, $dp$. Coefficients in $dp$ relative to the various variables are extracted and placed into a second matrix. It is not easy to predict how large this matrix will be. It might not be square. In the simple case of $n = 2$ it is square and its determinant is a multiple of the resultant of the original system. For almost all interesting problems, more is needed. Using a third matrix, a fourth matrix is extracted whose determinant is a multiple of the resultant. How this is done is the KSY idea. There are many possible fourth matrices. Theoretically one now simply computes the determinant of the fourth matrix. However in real examples this is usually impossible – the answer would be horrendously large. The EDF idea, and several other ideas, often provide a feasible method.

The crucial Kapur-Saxena-Yang (KSY) idea seems to have not been noticed in some research communities. Many researchers still try to use Gröbner bases even though that technique frequently fails, especially when there are parameters. In [7] it was shown than when there are parameters, Dixon-EDF is often enormously superior. We find that to be true in this work. In Appendix 1 we present the commands that we used to run Maple and Magma for the first computation below.

## 3 The Standard 2D and 3D Cases

In two dimensions suppose the sensor is at point $(x, y)$. Assume there are three distinct points on A. We have our measured distances to B, $t_1, t_2, t_3$ and previously known distances between the points on A, $d_1, d_2, d_3$. Let the

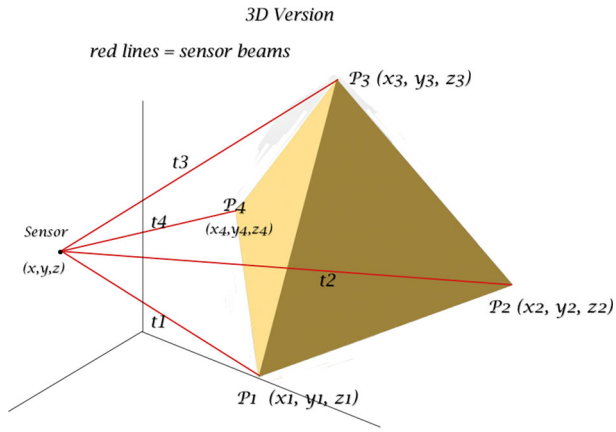


**Fig. 1** Sensor and object in 3D, showing four points on yellow object (colour figure online)

coordinates of the three points be $(x_1, y_1)$, $(x_2, y_2)$, $(x_3, y_3)$. If A = B we get six equations (set each to 0):

$$(x - x_1)^2 + (y - y_1)^2 - t_1^2$$
$$(x - x_2)^2 + (y - y_2)^2 - t_2^2$$
$$(x - x_3)^2 + (y - y_3)^2 - t_3^2$$
$$(x_1 - x_2)^2 + (y_1 - y_2)^2 - d_1^2$$
$$(x_1 - x_3)^2 + (y_1 - y_3)^2 - d_2^2$$
$$(x_2 - x_3)^2 + (y_2 - y_3)^2 - d_3^2$$

We don't know the $x_i$ and $y_i$ so we have six equations in eight variables. However, by choice of coordinate system we can assume $x_1 = y_1 = 0$ and $y_2 = 0$, so there are really five variables.[1] We compute the resultant very easily, in a fraction of a second with Dixon-EDF on Fermat [10,11]. Gröbner bases on Magma and Maple also succeed in a bit more time. The answer has 22 terms:

$$d_1^2 t_3^4 + d_3^2 t_2^2 t_3^2 - d_2^2 t_2^2 t_3^2 - d_1^2 t_2^2 t_3^2 - d_3^2 t_1^2 t_3^2 + d_2^2 t_1^2 t_3^2 - d_1^2 t_1^2 t_3^2 - d_1^2 d_3^2 t_3^2$$
$$-d_1^2 d_2^2 t_3^2 + d_1^4 t_3^2 + d_2^2 t_2^4 - d_3^2 t_1^2 t_2^2 - d_2^2 t_1^2 t_2^2 + d_1^2 t_1^2 t_2^2 - d_2^2 d_3^2 t_2^2 + d_2^4 t_2^2$$
$$-d_1^2 d_2^2 t_2^2 + d_3^2 t_1^4 + d_3^4 t_1^2 - d_2^2 d_3^2 t_1^2 - d_1^2 d_3^2 t_1^2 + d_1^2 d_2^2 d_3^2$$

The Maple and Magma commands used to run this problem are in Appendix 1.

In three dimensions we need four points; see Fig. 1. There are now six mutual distances and four $t_i$, yielding ten equations just like the six above. After assuming $x_1 = y_1 = z_1 = y_2 = z_2 = z_3 = 0$, there are nine variables to eliminate. In 2018 Dixon-EDF took 5.2 s and 20 megabytes of RAM. With the 2019 block decomposition idea [12] this is reduced to 0.31 s and 6 megabytes of RAM. All subsequent references to Dixon-EDF timings will include the 2019 block decomposition method. Maple running Faugere's FGb package [4] takes 17 s and 2.2 gigabytes of RAM. Magma took 98 s and 132 megabytes of RAM. The resultant has 130 terms.

## 4 Flexible Objects

Motivated by airplanes that can flex their wings, we consider two cases: planes in which the wings rotate up and down, pivoting around the fuselage, and those in which the wings are swept back, maintaining the same altitude (Fig. 2).

It is not hard to derive equations for these cases. A fifth point is needed, as not all of the six mutual distances among four points remain fixed. We may assume $x_1 = y_1 = z_1 = y_2 = z_2 = y_3 = 0$. In the first case we assume

[1] If we don't think of this and leave in $x_1$, $y_1$, $y_2$, Dixon-EDF still works fine but takes longer.

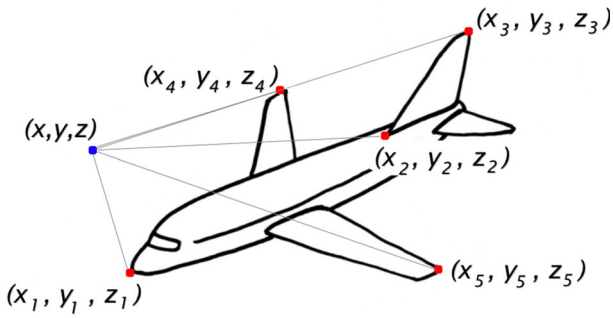


**Fig. 2** Sensor and airplane, showing five points. Points four and five pivot

that $x_5 = x_4$, $y_5 = -y_4$, $z_5 = z_4$. We do not assume that the wing tips follow a circular path. There are then ten equations:

$$
\begin{aligned}
&z^2 + y^2 + x^2 - t_1^2,\\
&z^2 + y^2 + x^2 - 2x_2x + x_2^2 - t_2^2,\\
&z^2 - 2z_3z + y^2 + x^2 - 2x_3x + z_3^2 + x_3^2 - t_3^2,\\
&z^2 - 2z_4z + y^2 - 2y_4y + x^2 - 2x_4x + z_4^2 + y_4^2 + x_4^2 - t_4^2,\\
&z^2 - 2z_4z + y^2 + 2y_4y + x^2 - 2x_4x + z_4^2 + y_4^2 + x_4^2 - t_5^2,\\
&x_2^2 - d_1^2, z_3^2 + x_3^2 - 2x_2x_3 + x_2^2 - d_2^2, z_3^2 + x_3^2 - d_3^2,\\
&z_4^2 + y_4^2 + x_4^2 - 2x_2x_4 + x_2^2 - d_4^2, z_4^2 + y_4^2 + x_4^2 - d_5^2
\end{aligned}
$$

In this first case, Dixon-EDF finishes in 0.25 s, using 7 meg of RAM. The resultant has 2373 terms. Maple with FGb takes 9 minutes and 6.8 gig of RAM. Magma did not finish in 1 h.

In the second case, the equations are very similar but $z_4 = z_5$ is now constant; we rename it $d_5$ (which is not what $d_5$ meant before). The tenth equation is dropped and there are nine equations. Dixon-EDF finishes in 1.0 s, using 77 meg of RAM. The resultant has 74,668 terms. Maple with FGb crashed after 19 h and 62 gig of RAM. Magma did not finish in 7 h.

Further generalizations are possible. Another flexible configuration, not immediately identifiable as an airplane, may be formed by taking points 1, 2, and 3 to be fixed in a plane. Let point 4 be joined to 1 and 3 by rigid rods but free to pivot. Let point 5 be joined to points 1 and 2 by rigid rods but free to pivot. Attach a rod between points 4 and 5. There are thirteen parameters: eight distances $d_i$ and five measured $t_i$. There are twelve variables. The resultant takes Dixon-EDF 13 to 30 minutes to compute (depending on the variation of EDF used), using 2 gig of RAM, and has 189,441 terms. Maple with FGb crashed after 9 h and 60 gig of RAM. Magma was killed after 100 h.

One can also imagine scenarios in which five points are needed because one may not be visible to the sensor at times.

## 5 Transmitter–Receiver Model

Suppose the sensor does not send out beams but can only receive. It can detect the total time, hence distance, the beam travels from a separate transmitter after bouncing off the object. These situations may be referred to as echo-based sensors, bistatic sonar, or bistatic radar [5,15]. We consider the general case in Fig. 3.

$d_1 + d_2 = p$, a known time (or distance). Each $d_i$ is a simple radical expression, so we must square twice to produce a polynomial. In Fig. 4 we consider the two dimensional case of four points on an object. The coordinate system on the object is known, so we may assume that $x_1 = y_1 = y_2 = 0$. Line P and value $a$ will be explained below. We compute the obvious four equations. They are very messy. For example, the one for point 3 has 47 terms.

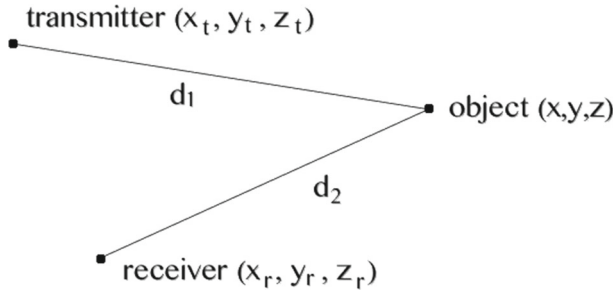


**Fig. 3** Transmitter and receiver

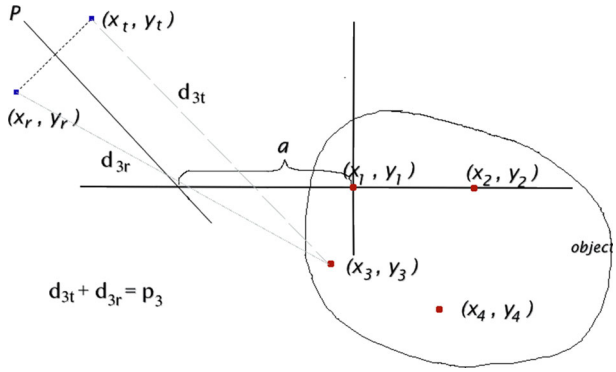


**Fig. 4** Transmitter, receiver, and four points on an object

In part it is

$$y_t^4 - 4y_3y_t^3 + 2x_t^2y_t^2 - 4x_3x_ty_t^2 - 2y_r^2y_t^2 + \cdots + 4p_3^2x_3x_r - 4p_3^2y_3^2 - 4p_3^2x_3^2 + p_3^4$$

To eliminate the four variables $x_t, y_t, x_r, y_r$ we would need a fifth equation.

A direct attack seems hopeless so we look for algebraic simplifications. First we observe that a substitution greatly simplifies the equations:

$$x_t = (u_1 + u_2)/2,\ y_t = (v_1 + v_2)/2,\ x_r = (u_1 - u_2)/2,\ y_r = (v_1 - v_2)/2$$

The equation for point 3 (or point 4) now contains only 19 terms. However, we still need a fifth equation and a fifth point. The resulting system is too large to be solved on a computer with 132 gig of RAM.

As before we set $x_1 = y_1 = y_2 = 0$. Next we observe that a big simplification would result if we knew that $s \equiv u_1u_2 + v_1v_2$ were 0. If we mod out the four equations by $s$ we obtain

$$\begin{aligned}
&v_1^4 + u_2^2v_1^2 + u_1^2v_1^2 - p_1^2v_1^2 + u_1^2u_2^2,\\
&p_2^2v_1^4 - 4x_2^2u_2^2v_1^2 + p_2^2u_2^2v_1^2 + p_2^2u_1^2v_1^2 - 4p_2^2x_2u_1v_1^2 + 4p_2^2x_2^2v_1^2 - p_2^4v_1^2 + p_2^2u_1^2u_2^2,\\
&p_3^2v_1^4 - 4p_3^2y_3v_1^3 - 4x_3^2u_2^2v_1^2 + p_3^2u_2^2v_1^2 + p_3^2u_1^2v_1^2 - 4p_3^2x_3u_1v_1^2 + 4p_3^2y_3^2v_1^2 + 4p_3^2x_3^2v_1^2\\
&\quad - p_3^4v_1^2 + 8x_3y_3u_1u_2^2v_1 - 4y_3^2u_1^2u_2^2 + p_3^2u_1^2u_2^2,\\
&p_4^2v_1^4 - 4p_4^2y_4v_1^3 - 4x_4^2u_2^2v_1^2 + p_4^2u_2^2v_1^2 + p_4^2u_1^2v_1^2 - 4p_4^2x_4u_1v_1^2 + 4p_4^2y_4^2v_1^2\\
&\quad + 4p_4^2x_4^2v_1^2 - p_4^4v_1^2 + 8x_4y_4u_1u_2^2v_1 - 4y_4^2u_1^2u_2^2 + p_4^2u_1^2u_2^2
\end{aligned}$$

Fortuitously, the variable $v_2$ disappears (or whichever variable is set to have highest rank). The four equations are enough to eliminate $u_1, u_2, v_1$. This system is solvable by Dixon-EDF in about 13 minutes. The resultant $res$ has 358,560 terms.

How can we justify the assumption that $s = u_1u_2 + v_1v_2 = 0$? In terms of the original variables, $s = 0$ means $x_t^2 + y_t^2 = x_r^2 + y_r^2$. This says that the transmitter and receiver are the same distance from the origin, or equivalently, that the origin lies on the perpendicular bisector of the line connecting them. This is shown in Fig. 4 as line P. There is no reason for the origin to be on line P. However, barring an extremely unlikely singularity, there is a number $a$ so that the origin of the coordinate system shifted by $a$ will be on P; see Fig. 4.

To now perform the A = B test on an actual example, we would obtain numerical values $x_i^*$, $y_i^*$ and $p_i^*$ for parameters. All the numerical $y_i^*$ and $p_i^*$ would be plugged in to $res$, then the $x_i$ would be replaced with $(x_i^* + a)$. The resulting polynomial in $a$ would be solved numerically. The test for A = B is that there is a reasonable real solution to this equation. If needed, more such tests could be done by permuting the four points. Further, one could check that the $a$ values produce points that lie on the perpendicular bisector P.

A three dimensional generalization is immediate. We have transmitter and receiver at $(x_t, y_t, z_t)$, $(x_r, y_r, z_r)$. We have a substitution generalizing the above, adding $z_t = (w_1 + w_2)/2$, $z_r = (w_1 - w_2)/2$. We need five points $(x_i, y_i, z_i)$. If the object is three-dimensional, so that at least two points have a $z$ coordinate in its preferred coordinate system, the system of six equations seems hopelessly complex. However, if we assume the object is flat, then in the preferred coordinate system there are no $z$ coordinates. The equations for that are in Appendix 2. Work is continuing on this case.

## 6 Appendix 1

The Maple-FGb commands for the first example:

```
    |\^/|     Maple 2015 (X86 64 LINUX)
._|\|   |/|_.Copyright(c) Maplesoft,a division of Waterloo Maple Inc
 \  MAPLE  /  All rights reserved. Maple is a trademark of
 <____ ____>  Waterloo Maple Inc.
      |       Type ? for help.

with(FGb):
p := 0;
v1 := [ x, y, x2, x3, y3 ];
v2 := [ t1,t2,t3,d1,d2,d3 ];
sys := [y^2 + x^2 - t1^2 ,
        y^2 + x^2 - 2*x2*x + x2^2 - t2^2 ,
        y^2 - 2*y3*y + x^2 - 2*x3*x + y3^2 + x3^2 - t3^2 ,
        x2^2 - d1^2 ,
        y3^2 + x3^2 - d2^2 ,
        y3^2 + x3^2 - 2*x2*x3 + x2^2 - d3^2 ];
l:=fgb_gbasis_elim(sys,p,v1,v2,{"step"=8,"verb"=3,"index"=40000000});
```

Magma commands for the first example:

```
Magma V2.21-8  May 1 2018 3:26:28 on math01 [Seed = 2343837211]
Type ? for help.  Type <Ctrl>-D to quit.
 Q:=RationalField();
 F<t1,t2,t3,d1,d2,d3> := FunctionField(Q,6);
 R<x, y, x3, y3, x2> := PolynomialRing(F,5, "elim", [1,2,3,4]);
 I := Ideal ([y^2 + x^2 - t1^2 ,
        y^2 + x^2 - 2*x2*x + x2^2 - t2^2 ,
        y^2 - 2*y3*y + x^2 - 2*x3*x + y3^2 + x3^2 - t3^2 ,
        y3^2 + x3^2 - d2^2 ,
        y3^2 + x3^2 - 2*x2*x3 + x2^2 - d3^2]);
 time G := GroebnerBasis(I);
```

Note that we dropped the equation $x_2^2 - d_1^2$. In the answer, $G$, replace $x_2$ with $d_1$.

## 7 Appendix 2

Let $pxr$ be the distance between the transmitter and receiver. There are five points on the object. In terms of the $x_i$, $y_i$ and $z_i$ coordinates the six equations are:

$$
\begin{aligned}
&(xr - xt)^2 + (yr - yt)^2 + (zr - zt)^2 - pxr,\\
&p_1^4 - 2p_1^2(xr^2 + yr^2 + zr^2 + xt^2 + yt^2 + zt^2) + (xr^2 + yr^2 + zr^2 - xt^2 - yt^2 - zt^2)^2,\\
&p_2^4 - 2p_2^2((xr - x_2)^2 + yr^2 + zr^2 + (xt - x_2)^2 + yt^2 + zt^2) + ((xr - x_2)^2\\
&\quad + yr^2 + zr^2 - (xt - x_2)^2 - yt^2 - zt^2)^2,\\
&p_3^4 - 2p_3^2((xr - x_3)^2 + (yr - y_3)^2 + (zr - z_3)^2 + (xt - x_3)^2 + (yt - y_3)^2 + (zt - z_3)^2)\\
&\quad + ((xr - x_3)^2 + (yr - y_3)^2 + (zr - z_3)^2 - (xt - x_3)^2 - (yt - y_3)^2 - (zt - z_3)^2)^2,\\
&p_4^4 - 2p_4^2((xr - x_4)^2 + (yr - y_4)^2 + (zr - z_4)^2 + (xt - x_4)^2 + (yt - y_4)^2 + (zt - z_4)^2)\\
&\quad + ((xr - x_4)^2 + (yr - y_4)^2 + (zr - z_4)^2 - (xt - x_4)^2 - (yt - y_4)^2 - (zt - z_4)^2)^2,\\
&p_5^4 - 2p_5^2((xr - x_5)^2 + (yr - y_5)^2 + (zr - z_5)^2 + (xt - x_5)^2 + (yt - y_5)^2 + (zt - z_5)^2)\\
&\quad + ((xr - x_5)^2 + (yr - y_5)^2 + (zr - z_5)^2 - (xt - x_5)^2 - (yt - y_5)^2 - (zt - z_5)^2)^2
\end{aligned}
$$

If those polynomials are expanded, they are extremely messy.

When all the $z_i$ are set to 0 and the $x_r$, $y_r$, $z_r$, $x_t$, $y_t$, $z_t$ are replaced by $u_i$, $v_i$ and $w_i$, we get

$$
\begin{aligned}
&v_2^2w_1^2 + u_2^2w_1^2 - pxrw_1^2 + v_1^2v_2^2 + 2u_1u_2v_1v_2 + u_1^2u_2^2,\\
&\quad - p_1^2w_1^4 - p_1^2v_2^2w_1^2 - p_1^2v_1^2w_1^2 - p_1^2u_2^2w_1^2 - p_1^2u_1^2w_1^2 + p_1^4w_1^2 - p_1^2v_1^2v_2^2 - 2p_1^2u_1u_2v_1v_2 - p_1^2u_1^2u_2^2,\\
&\quad - p_2^2w_1^4 - p_2^2v_2^2w_1^2 - p_2^2v_1^2w_1^2 + 4x_2^2u_2^2w_1^2 - p_2^2u_2^2w_1^2 - p_2^2u_1^2w_1^2 + 4p_2^2x_2u_1w_1^2 - 4p_2^2x_2^2w_1^2\\
&\qquad + p_2^4w_1^2 - p_2^2v_1^2v_2^2 - 2p_2^2u_1u_2v_1v_2 - p_2^2u_1^2u_2^2,\\
&\quad - p_3^2w_1^4 + 4y_3^2v_2^2w_1^2 - p_3^2v_2^2w_1^2 + 8x_3y_3u_2v_2w_1^2 - p_3^2v_1^2w_1^2 + 4p_3^2y_3v_1w_1^2 + 4x_3^2u_2^2w_1^2 - p_3^2u_2^2w_1^2 - p_3^2u_1^2w_1^2\\
&\qquad + 4p_3^2x_3u_1w_1^2 - 4p_3^2y_3^2w_1^2 - 4p_3^2x_3^2w_1^2 + p_3^4w_1^2 - p_3^2v_1^2v_2^2 - 2p_3^2u_1u_2v_1v_2 - p_3^2u_1^2u_2^2,\\
&\quad - p_4^2w_1^4 + 4y_4^2v_2^2w_1^2 - p_4^2v_2^2w_1^2 + 8x_4y_4u_2v_2w_1^2 - p_4^2v_1^2w_1^2 + 4p_4^2y_4v_1w_1^2 + 4x_4^2u_2^2w_1^2 - p_4^2u_2^2w_1^2 - p_4^2u_1^2w_1^2\\
&\qquad + 4p_4^2x_4u_1w_1^2 - 4p_4^2y_4^2w_1^2 - 4p_4^2x_4^2w_1^2 + p_4^4w_1^2 - p_4^2v_1^2v_2^2 - 2p_4^2u_1u_2v_1v_2 - p_4^2u_1^2u_2^2,\\
&\quad - p_5^2w_1^4 + 4y_5^2v_2^2w_1^2 - p_5^2v_2^2w_1^2 + 8x_5y_5u_2v_2w_1^2 - p_5^2v_1^2w_1^2 + 4p_5^2y_5v_1w_1^2 + 4x_5^2u_2^2w_1^2 - p_5^2u_2^2w_1^2 - p_5^2u_1^2w_1^2\\
&\qquad + 4p_5^2x_5u_1w_1^2 - 4p_5^2y_5^2w_1^2 - 4p_5^2x_5^2w_1^2 + p_5^4w_1^2 - p_5^2v_1^2v_2^2 - 2p_5^2u_1u_2v_1v_2 - p_5^2u_1^2u_2^2.
\end{aligned}
$$

## References


1. Buse, L., Elkadi, M., Mourrain, B.: Generalized resultants over unirational algebraic varieties. J. Symb. Comput. **29**, 515–526 (2000)
2. Cox, D., Little, J., O'Shea, D.: Using Algebraic Geometry. Graduate Texts in Mathematics. Springer, New York (1998)
3. Dixon, A.L.: The eliminant of three quantics in two independent variables. Proc. Lond. Math. Soc. **6**, 468–478 (1908)
4. Faugere, J.-C.: A new efficient algorithm for computing Gröbner bases (F4). J. Pure Appl. Algebra **139**, 61–88 (1999)
5. Griffiths, H.: Bistatic and multistatic radar. In: IEEE Military Radar Seminar (2004)
6. Kapur, D., Saxena, T., Yang, L.: Algebraic and geometric reasoning using Dixon resultants. In: Proceedings of the International Symposium on Symbolic and Algebraic Computation (1994). https://doi.org/10.1145/190347.190372
7. Lewis, R.H.: Dixon-EDF: The premier method for solution of parametric polynomial systems. In: Kotsireas, I. S., Martinez-Moro, E. (eds) Applications of Computer Algebra, Kalamata, Greece. Springer Proceedings in Mathematics & Statistics, vol. 198 (2017)
8. Lewis, R.H.: Heuristics to accelerate the Dixon resultant. Math. Comput. Simul. **77**(4), 400–407 (2008)
9. Lewis, R.H., Stiller, P.: Solving the recognition problem for six lines using the dixon resultant. Math. Comput. Simul. **49**, 203–219 (1999)
10. Lewis, R.H.: Computer algebra system. Fermat. http://home.bway.net/lewis/

11. Lewis, R.H.: Fermat code for Dixon-EDF. http://home.bway.net/lewis/dixon
12. Lewis, R.H.: New heuristics and extensions of the Dixon resultant for solving polynomial systems. Talk at: Applications of Computer Algebra, Montreal, Canada, 16–20 July 2019
13. Stiller, P.: Symbolic computation of object/image equations. In: Proceedings of the International Symposium on Symbolic and Algebraic Computation, pp. 359–364. ACM Press, New York (1997)
14. Sturmfels, B.: Solving systems of polynomial equations. In: CBMS Regional Conference Series in Mathematics, vol. 97. American Mathematical Society (2003)
15. Willis, N.J.: Bistatic Radar. SciTech Publishing, Raleigh (2005)